Minerando métricas DORA em repositórios open-source: o quanto podemos confiar nos *proxies*?

Luisa Clara de Paula Lara Silva, Matheus Gaston, Mirelly Alvarenga

Curso de Engenharia de Software
Laboratório de Experimentação de Software

Resumo. *Este estudo minera métricas DORA de 300 repositórios open-source populares que usam GitHub Actions, em uma janela de 12 meses. Releases, commits, tags e execuções de workflows são empregados como proxies de eventos de produção. Os resultados mostram grande variação entre repositórios e, principalmente, sensibilidade à definição operacional: entre 21,7% e 37,7% dos repositórios mudam de categoria entre três configurações plausíveis. A concordância ponderada varia de 0,432 a 0,683. A taxa de retrabalho tem mediana de 14,3% e é relacionada, mas conceitualmente distinta, do CFR baseado em releases. A frequência de deploy não se correlaciona com o CFR de CI ($\rho = 0{,}01$), mas se correlaciona com o CFR de entrega ($\rho = 0{,}60$), em grande parte por efeito do prazo de sete dias da definição. Entre as características dos repositórios, a idade é a mais associada ao desempenho, com projetos mais antigos publicando menos e com lead time maior; nenhuma característica se associa às métricas de estabilidade.*

1 Introdução

As métricas DORA (*DevOps Research and Assessment*) tornaram-se uma referência para medir o desempenho da entrega de software. Elas descrevem a entrega em duas dimensões: velocidade, medida pela frequência de *deploy* e pelo *lead time* das mudanças; e estabilidade, medida pela taxa de falha das mudanças (*change failure rate*, CFR) e pelo tempo de recuperação [1, 2]. Em 2024, a DORA passou a tratar também a *deployment rework rate*, proporção de implantações não planejadas destinadas a corrigir problemas.

Em organizações, essas métricas podem ser obtidas de sistemas internos que registram implantações e incidentes. Em projetos open-source, porém, esses eventos normalmente não são observáveis. O GitHub registra releases, tags, commits e execuções de integração contínua (CI), exigindo o uso de *proxies*. Neste trabalho, por exemplo, uma release representa um deploy e uma execução de CI malsucedida representa uma falha.

Essas aproximações introduzem ameaças de construto: release não é necessariamente deploy; falha de CI não é necessariamente falha em produção; e estratégias como *rebase*

e *squash merge* podem distorcer as datas usadas no *lead time*. Portanto, duas definições razoáveis podem levar a classificações diferentes para o mesmo repositório.

O objetivo deste estudo é minerar as métricas DORA de repositórios open-source populares que usam GitHub Actions, em uma janela de 12 meses, e avaliar o quanto os resultados são confiáveis. As questões de pesquisa são:

- **RQ01.** Qual a frequência de deploy dos repositórios?
- **RQ02.** Qual o tempo entre um commit e seu respectivo deploy?
- **RQ03.** Qual a taxa de falha das mudanças entregues?
- **RQ04.** Qual o tempo de recuperação após falha de CI/CD?
- **RQ05.** Frequência de deploy e CFR estão associados?
- **RQ06.** Quais características estão associadas ao desempenho?
- **RQ07.** A classificação depende da definição operacional?
- **RQ08.** Qual a taxa de retrabalho e como ela difere do CFR?

As hipóteses foram registradas antes da análise. Em síntese, esperava-se frequência mediana de releases, *lead time* de semanas, CFR entre 10% e 25%, recuperação em horas, associação fraca entre velocidade e estabilidade, efeitos pequenos das características dos repositórios, sensibilidade superior a 30% nas classificações e *rework rate* entre 20% e 35%. A cópia datada preservada no repositório contém a formulação integral de H1–H8.

2 Metodologia

2.1 Amostra e coleta

A unidade de análise é o repositório. A amostra contém 300 repositórios open-source populares que utilizam GitHub Actions. A janela de observação vai de 30/09/2025 a 30/09/2026. O conjunto analisado reúne 17.859 releases, 1.361.066 commits e 491.249 execuções válidas de workflows. A coleta e o processamento foram automatizados, e os artefatos intermediários foram preservados para auditoria e replicação.

2.2 Definições operacionais

A Tabela 1 resume os proxies usados. As análises descritivas apresentam mediana e intervalo interquartil (IQR), adequados às distribuições assimétricas observadas.

Para a análise de sensibilidade (RQ07), foram montadas três combinações: C1 usa releases, *lead time* (a) e CFR (a); C2 inclui releases e pré-releases, *lead time* (b) e CFR (b); C3 usa tags, *lead time* (b) e CFR (a). Cada repositório foi classificado nas categorias Elite, High, Medium ou Low. Para cada par, calculou-se a proporção que mudou de categoria e o kappa de Cohen ponderado linearmente.

A RQ08 aplica a mesma heurística final de release corretiva usada na RQ03(b). A diferença está no objeto contado: o CFR atribui a falha à release anterior quando uma

Tabela 1: Definições operacionais das métricas.

Métrica	Unidade observável	Definição resumida
RQ01	releases, pré-releases e tags	quantidade de eventos por semana na janela observada
RQ02(a)	release	intervalo entre a publicação e o commit mais antigo associado
RQ02(b)	commit	intervalo entre cada commit e a release que o contém
RQ03(a)	workflow	execuções malsucedidas de push no branch principal sobre execuções válidas
RQ03(b)	release	release seguida em até sete dias por uma versão apenas de patch
RQ04	episódio de CI	tempo entre uma execução malsucedida e a próxima execução bem-sucedida
RQ08	release	releases classificadas como corretivas sobre o total de releases

correção chega em até sete dias; o *rework rate* conta a própria release corretiva, sem restringir sua publicação a essa janela.

3 Resultados

3.1 RQ01–RQ04: panorama das quatro métricas

A Tabela 2 reúne os principais resultados. A frequência baseada somente em releases tem mediana de 0,35 evento por semana (IQR 0,17–0,77). Quando pré-releases são incluídas, a mediana sobe para 0,38; com tags, chega a 0,45. A Figura 1 mostra a assimetria e a sensibilidade à unidade de deploy.

Tabela 2: Síntese dos resultados de RQ01–RQ04.

RQ	Medida	Mediana	Q1	Q3
RQ01	releases por semana	0,35	0,17	0,77
RQ01	release + pré-release	0,38	0,19	0,88
RQ01	tags por semana	0,45	0,23	1,02
RQ02	lead time (a), dias	10,6	2,2	33,3
RQ02	lead time (b), dias	8,1	2,0	29,6
RQ03	CFR (a), %	5,7	2,0	13,5
RQ03	CFR (b), %	12,5	0,0	30,2
RQ04	recuperação, horas	3,1	0,9	17,4

O *lead time* (a) apresenta mediana de 10,6 dias, contra 8,1 dias na variante (b). A correlação de Spearman entre as variantes é 0,72, mas apenas 55,2% dos repositórios permanecem na mesma categoria DORA. Logo, associação alta entre valores não implica estabilidade da classificação.

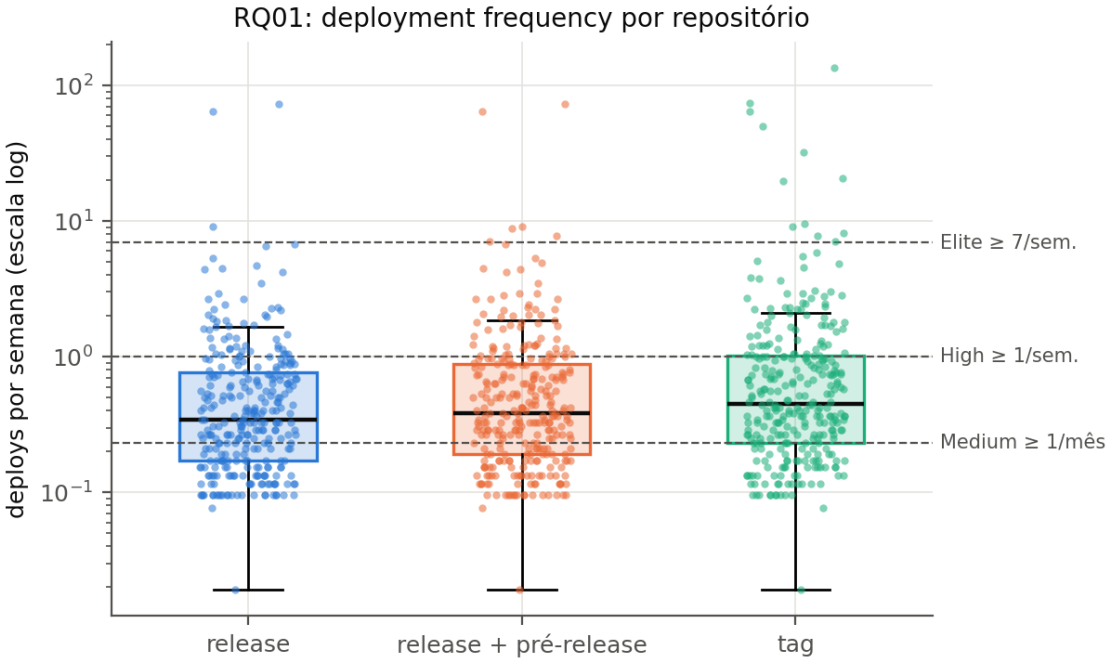


Figura 1: Distribuição da frequência de deploy sob três proxies.

Na RQ03, o CFR (a) tem mediana de 5,7%, enquanto o CFR (b) tem mediana de 12,5%. A correlação entre os proxies é praticamente nula ($\rho = 0{,}06$, $p = 0{,}27$), e somente 49,7% dos repositórios recebem a mesma categoria. A Figura 2 evidencia que os proxies capturam fenômenos diferentes. Houve ainda 400 releases censuradas na borda da janela.

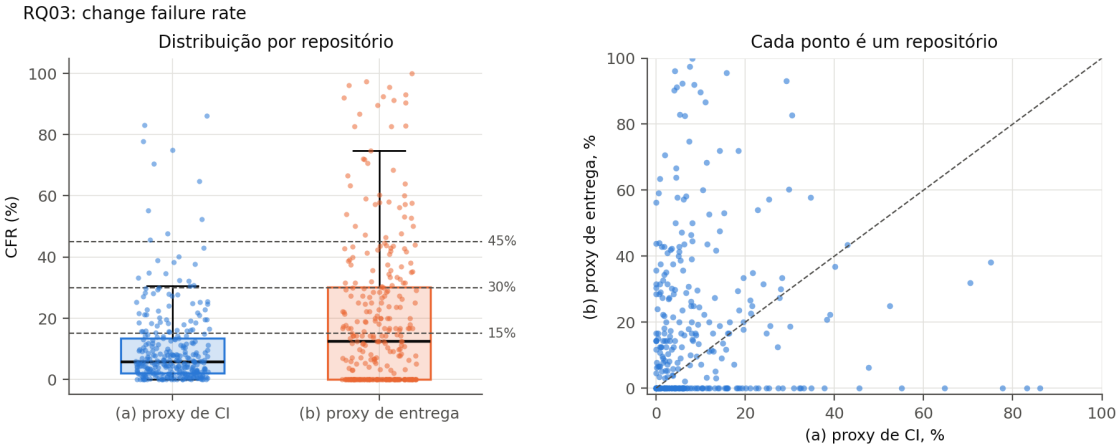


Figura 2: Distribuição e comparação dos dois proxies de CFR.

Na RQ04, a mediana do tempo de recuperação é 3,1 horas, com IQR de 0,9–17,4 horas. A métrica pôde ser calculada para 276 repositórios. Dos episódios observados, 204 (1,5%) permaneceram censurados por não haver uma execução bem-sucedida posterior dentro da janela.

3.2 RQ05: frequência de deploy e taxa de falha

A Tabela 3 traz a correlação de Spearman entre a frequência de deploy (releases por semana) e cada variante de CFR, com intervalo de confiança de 95% por *bootstrap* percentil (2.000 reamostragens dos repositórios). Com o CFR (a), não há relação monotônica ($\rho = 0{,}01$, $p = 0{,}86$), e o intervalo descarta uma relação mais que fraca em qualquer direção. Com o CFR (b), a correlação é positiva e forte ($\rho = 0{,}60$, $p < 0{,}001$). A Figura 3 mostra os dois casos: a mediana do CFR (a) por decil de frequência fica entre 4% e 10% em toda a faixa, enquanto a do CFR (b) vai de 0% nos decis mais lentos a mais de 70% no mais rápido.

Tabela 3: Correlação de Spearman entre frequência de deploy e taxa de falha.

Taxa de falha	n	ρ	IC 95% de ρ	p
CFR (a), *proxy* de CI	300	0,010	[−0,10; 0,12]	0,857
CFR (b), *proxy* de entrega	300	0,602	[0,51; 0,68]	<0,001
Rework rate (controle do CFR (b))	300	0,195	[0,07; 0,31]	<0,001

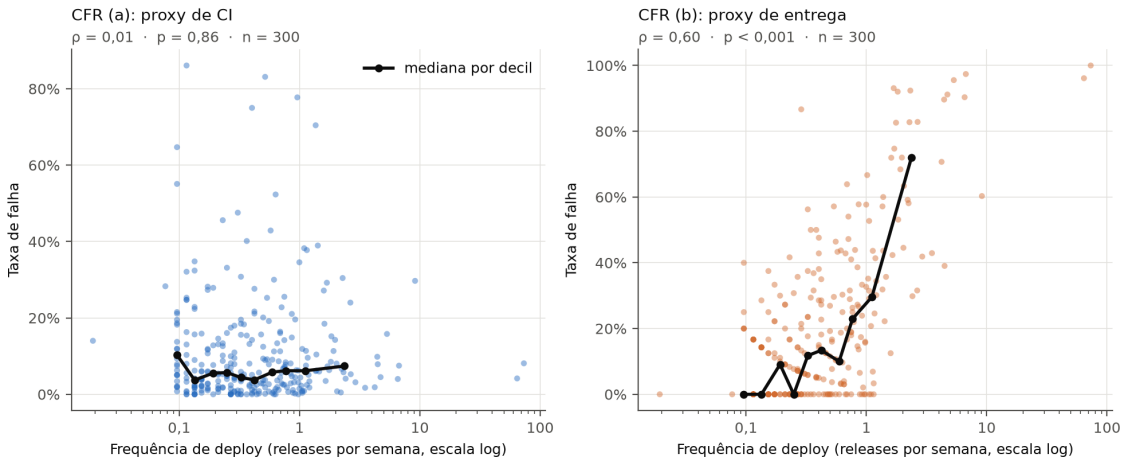


Figura 3: Frequência de deploy (escala log) contra CFR (a), à esquerda, e CFR (b), à direita. A linha liga a mediana do CFR em cada decil de frequência.

Duas verificações separam o comportamento dos projetos do efeito da definição. Restringir a análise aos 215 repositórios com pelo menos 10 releases avaliadas, para evitar CFRs (b) discretos como 0%, 50% ou 100%, mantém $\rho = 0{,}55$ para o CFR (b) e $\rho = 0{,}11$ ($p = 0{,}10$) para o CFR (a). Já a *rework rate*, que usa a mesma heurística de release corretiva mas não depende do prazo de sete dias, tem correlação bem menor com a frequência ($\rho = 0{,}20$), e os intervalos das duas não se sobrepõem. Logo, a maior parte da associação do CFR (b) vem do prazo fixo: quem publica uma vez por mês raramente tem outra release em sete dias, e quem publica várias vezes por semana quase sempre tem.

3.3 RQ06: características associadas ao desempenho

Foram comparadas as quatro métricas da configuração C1 (frequência, *lead time* (a), CFR (a) e recuperação) entre os grupos de quatro fatores: linguagem principal (seis

linguagens com pelo menos 15 repositórios e o grupo Outras, com 85) e quartis de estrelas, de contribuidores e de idade. O tipo do projeto não pôde ser usado: nenhum dos 60 repositórios rotulados na amostra-ouro está na amostra final, pois o sorteio foi feito sobre uma seleção anterior. Todos os fatores têm mais de dois grupos, então os 16 testes são de Kruskal-Wallis, com os p-valores corrigidos juntos por Holm. A Tabela 4 traz o tamanho de efeito $\varepsilon^2 = H/(n-1)$, interpretado pela escala de Tomczak e Tomczak [4], e o *Cliff's delta* (δ) entre os grupos extremos, interpretado por Romano et al. [3], ambos com intervalo de confiança de 95% por *bootstrap* percentil. Como ε^2 nunca é negativo, seu intervalo não inclui zero e não serve como teste; a significância é decidida pelo p corrigido.

Tabela 4: Kruskal-Wallis por fator e métrica, com p corrigido por Holm e tamanhos de efeito com IC 95%. Em negrito, os significativos após a correção. Nos quartis, δ compara Q4 com Q1; na linguagem, o grupo de maior mediana com o de menor.

Fator	Métrica	n	H	p	p (Holm)	ε^2 [IC 95%]	δ [IC 95%]
Linguagem	Frequência	300	6,31	0,390	1,000	0,021 [0,009; 0,086]	0,19 [0,00; 0,38] (TypeScript × Outras)
Linguagem	*Lead time* (a)	299	22,95	<0,001	**0,011**	0,077 [0,042; 0,166]	0,38 [0,08; 0,66] (C++ × Python)
Linguagem	CFR (a)	300	2,09	0,911	1,000	0,007 [0,006; 0,063]	0,05 [−0,27; 0,39] (TypeScript × C#)
Linguagem	Recuperação	276	5,65	0,463	1,000	0,021 [0,011; 0,085]	0,44 [0,08; 0,76] (C++ × Rust)
Estrelas	Frequência	300	11,81	0,008	0,089	0,040 [0,012; 0,099]	0,17 [−0,02; 0,35]
Estrelas	*Lead time* (a)	299	6,01	0,111	1,000	0,020 [0,004; 0,075]	0,07 [−0,12; 0,25]
Estrelas	CFR (a)	300	2,00	0,573	1,000	0,007 [0,001; 0,050]	0,05 [−0,13; 0,24]
Estrelas	Recuperação	276	1,72	0,632	1,000	0,006 [0,001; 0,050]	0,02 [−0,16; 0,21]
Contribuidores	Frequência	300	1,44	0,696	1,000	0,005 [0,001; 0,043]	−0,01 [−0,20; 0,18]
Contribuidores	*Lead time* (a)	299	35,54	<0,001	<**0,001**	0,119 [0,065; 0,201]	0,51 [0,35; 0,66]
Contribuidores	CFR (a)	300	2,45	0,484	1,000	0,008 [0,001; 0,051]	0,08 [−0,10; 0,27]
Contribuidores	Recuperação	276	5,41	0,144	1,000	0,020 [0,004; 0,076]	0,21 [0,03; 0,40]
Idade	Frequência	300	29,89	<0,001	<**0,001**	0,100 [0,048; 0,177]	−0,42 [−0,58; −0,25]
Idade	*Lead time* (a)	299	62,64	<0,001	<**0,001**	0,210 [0,142; 0,301]	0,68 [0,54; 0,81]
Idade	CFR (a)	300	1,09	0,779	1,000	0,004 [0,001; 0,043]	−0,09 [−0,27; 0,10]
Idade	Recuperação	276	12,11	0,007	0,084	0,044 [0,014; 0,110]	0,33 [0,15; 0,51]

Quatro das 16 associações resistem à correção (Figuras 4 e 5). A idade tem os maiores efeitos: o quartil mais novo (até 963 dias) publica 0,73 release por semana, contra 0,25 a 0,33 nos demais, e tem *lead time* (a) mediano de 2,3 dias, contra 28,7 dias no quartil mais antigo ($\delta = 0{,}68$, grande). O número de contribuidores também se associa ao *lead time*: 3,9 dias no quartil com até 31 contribuidores e 25,7 dias no quartil com mais de 177 ($\delta = 0{,}51$), sem diferença na frequência. Na linguagem, Python e TypeScript têm os menores *lead times* (6,3 e 6,5 dias) e C++ e Outras, os maiores (cerca de 22 dias). As estrelas não se associam a nenhuma métrica após a correção, e as métricas de estabilidade quase não variam: no CFR (a), todos os ε^2 ficam abaixo de 0,01.

Três ressalvas limitam essas conclusões. Os grupos C# e C++ têm só 15 repositórios cada; o δ de C++ contra Python no *lead time* depende de um deles, e seu intervalo vai de 0,08 a 0,66. O grupo Outras reúne linguagens muito diferentes. E idade e contribuidores estão correlacionados ($\rho = 0{,}45$), de modo que seus efeitos sobre o *lead time* podem ser, em parte, o mesmo. As comparações são observacionais e não estabelecem causa.

3.4 RQ07: sensibilidade das definições

A configuração-base C1 classifica 5,7% da amostra como Elite, 41,7% como High, 45,7% como Medium e 7,0% como Low. Assim, 87,3% dos repositórios ficam concentrados nas duas categorias intermediárias.

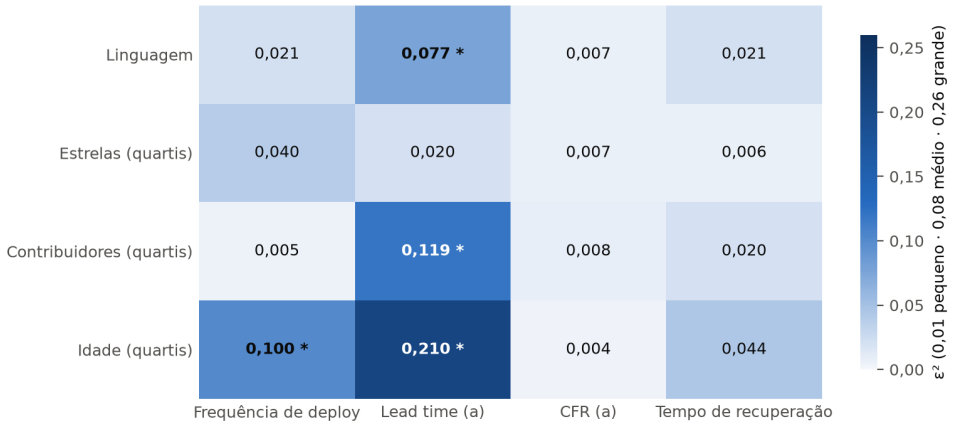


Figura 4: ε^2 de cada teste; o asterisco marca os significativos após a correção de Holm.

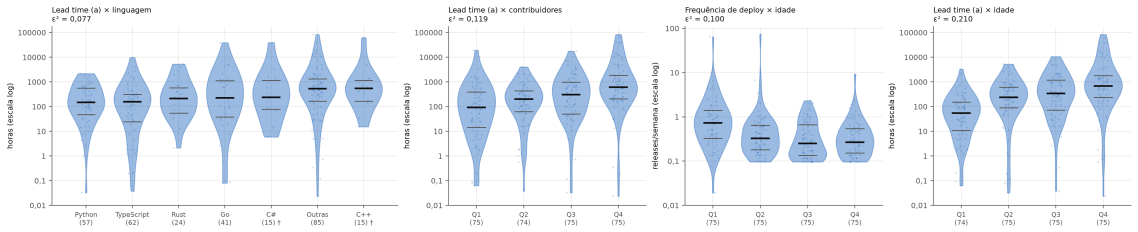


Figura 5: Violinos das métricas por grupo nas quatro associações significativas (escala log); a linha preta é a mediana e as cinzas, Q1 e Q3. Entre parênteses, o número de repositórios; † marca grupos com menos de 20.

A Tabela 5 confirma que a classificação depende da definição operacional. C1 e C2 produzem a maior instabilidade: 37,7% dos repositórios mudam de categoria e o kappa ponderado é 0,432. C2 e C3 apresentam a maior concordância, embora 21,7% ainda mudem de categoria.

Tabela 5: Mudança de categoria e concordância entre configurações.

Comparação	Mudaram	Mudança (%)	Kappa ponderado
C1 × C2	113/300	37,7	0,432
C1 × C3	90/300	30,0	0,594
C2 × C3	65/300	21,7	0,683

Das 113 mudanças entre C1 e C2, 107 ocorrem entre categorias adjacentes e nenhuma salta diretamente entre Elite e Low. A matriz de transição da Figura 6 mostra que a sensibilidade é substantiva, mas não arbitrária.

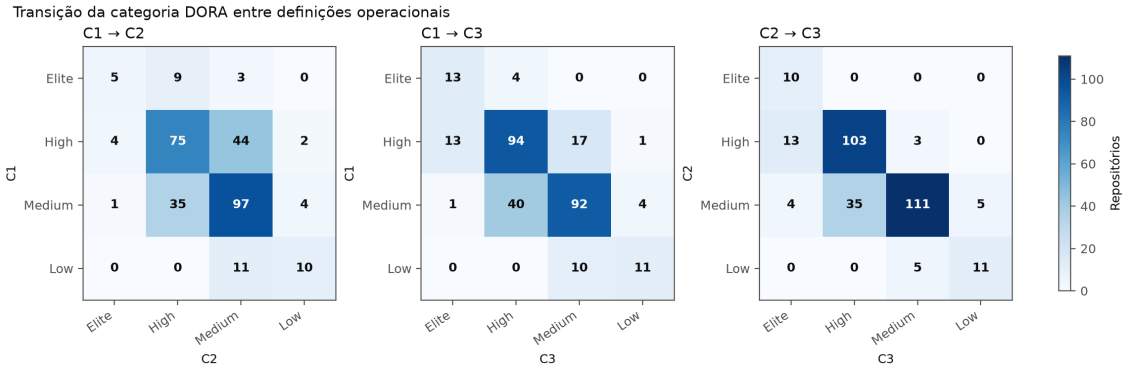


Figura 6: Matrizes de transição entre as classificações C1, C2 e C3.

3.5 RQ08: taxa de retrabalho

A taxa de retrabalho apresenta mediana de 14,3% (IQR 2,7–29,0%). No total agrupado, 4.509 das 17.859 releases foram classificadas como corretivas (25,2%). A comparação com o CFR (b) aparece na Tabela 6 e na Figura 7.

Tabela 6: Comparação entre *rework rate* e CFR (b).

Métrica	Mediana (%)	Q1 (%)	Q3 (%)
Rework rate	14,3	2,7	29,0
CFR (b)	12,5	0,0	30,2

As métricas têm correlação de Spearman de 0,697 ($p < 0{,}001$), mas não são intercambiáveis. O teste pareado de Wilcoxon não identifica diferença sistemática nas medianas ($p = 0{,}613$): o rework é maior em 98 repositórios, igual em 98 e menor em 104. O CFR (b) olha para a release que presumivelmente falhou; a taxa de retrabalho olha para a release

que presumivelmente corrige. Além disso, o CFR exige que a correção chegue em até sete dias, enquanto o rework não aplica essa restrição temporal.

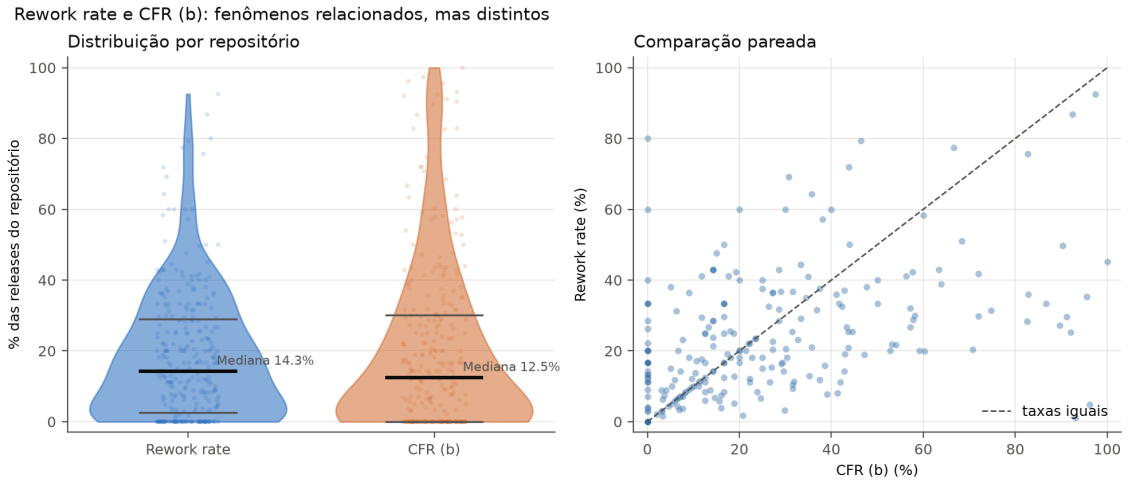


Figura 7: Distribuição e relação entre *rework rate* e CFR (b).

4 Discussão

Os resultados disponíveis sustentam parcialmente as hipóteses registradas. A frequência mediana baseada em releases é inferior a uma publicação por semana e tem forte assimetria, em linha com H1. Em H2, confirmou-se que o *lead time* por release tende a ser maior que o calculado por commit; porém, a relação não é sistemática: a variante (b) supera a (a) em 37% dos repositórios porque as variantes atribuem pesos diferentes a releases e commits. A diferença de classificação mostra que a escolha não é apenas técnica. H3 não foi sustentada para o CFR de CI, cuja mediana ficou abaixo da faixa esperada, e a associação quase nula entre os CFRs reforça que eles medem fenômenos distintos. H4 foi compatível com a recuperação mediana em poucas horas, embora exista uma cauda longa.

H7 foi parcialmente sustentada. A comparação C1–C2 supera o limiar esperado de 30% de mudanças e apresenta concordância moderada; C1–C3 atinge exatamente 30%, enquanto C2–C3 é mais estável. A troca envolvendo tags não foi, portanto, a mais instável. A maior mudança ocorreu na combinação que troca simultaneamente pré-releases, a ponderação do *lead time* e o proxy do CFR; por isso, o experimento mede sensibilidade conjunta, não o efeito causal isolado de cada escolha.

H8 não foi sustentada na mediana: a taxa de retrabalho de 14,3% ficou abaixo dos 20%–35% esperados e próxima do CFR (b). Ainda assim, a diferença conceitual permanece importante. Métricas com distribuições semelhantes podem atribuir significado distinto aos mesmos eventos.

H5 foi sustentada. Como previsto, o CFR (a) não se correlaciona com a frequência ($\rho = 0{,}01$), e o CFR (b) se correlaciona positivamente ($\rho = 0{,}60$). Pelo *proxy* de CI, os dados são compatíveis com a afirmação da DORA de que velocidade e estabilidade não são um *trade-off*: quem publica mais não quebra mais o *pipeline*. Eles não confirmam, porém, a versão mais forte da afirmação, de que as equipes mais rápidas também são as

mais estáveis; observa-se independência, não associação positiva. Pelo *proxy* de entrega, os dados parecem contradizer a DORA, mas as verificações da RQ05 indicam que a maior parte da correlação é um artefato do prazo de sete dias, que cai de 0,60 para 0,20 sem ele. O resíduo admite uma leitura que não é de instabilidade: em projetos que publicam com frequência, uma correção vira sua própria release, enquanto em projetos lentos ela entra na próxima versão planejada. Em ambos os casos, correlação não implica causalidade: tamanho, idade e maturidade do processo podem influir nas duas medidas ao mesmo tempo.

H6 foi sustentada em parte. A idade se comportou como esperado: repositórios mais antigos publicam menos e têm *lead times* maiores, com os maiores efeitos da análise. O número de contribuidores, ao contrário do previsto, não se associa à frequência nem à recuperação, e sim a *lead times* maiores. Na linguagem, TypeScript e Rust têm as maiores frequências medianas, na direção prevista, mas sem significância. A previsão de efeitos pequenos ($\varepsilon^2 < 0{,}06$) valeu para 12 dos 16 testes, mas três das quatro associações significativas têm efeito médio (ε^2 de 0,10 a 0,21). O *lead time* (a) concentra três desses quatro resultados; como ele parte do commit mais antigo da release, projetos antigos e grandes, com ramos de manutenção e séries paralelas de versões, tendem a inflá-lo, e parte do efeito pode ser da métrica, não do processo. Nenhuma característica se associa às métricas de estabilidade. Como só a configuração C1 foi testada, a robustez dessas associações às demais definições continua em aberto.

Em conjunto, RQ01–RQ04, RQ07 e RQ08 mostram que as conclusões descritivas gerais se mantêm: há grande heterogeneidade entre projetos, distribuições assimétricas e diferenças entre velocidade e estabilidade. Contudo, conclusões sobre a categoria DORA de um repositório e sobre o nível absoluto de CFR dependem fortemente da definição. Assim, os resultados devem ser interpretados como propriedades dos proxies escolhidos, não como medidas diretas de desempenho em produção.

5 Ameaças à validade

A principal ameaça de construto é a distância entre eventos do GitHub e eventos de produção. Releases e tags podem representar apenas publicação de pacotes; falhas de CI podem vir de testes instáveis ou infraestrutura; e uma mudança de patch nem sempre corrige defeito. A validação manual da heurística de releases corretivas reduz, mas não elimina, esse risco.

Quanto à validade interna, datas reescritas, paginação da API, dados ausentes e censura nas bordas da janela podem alterar as métricas. Foram usados critérios determinísticos, cache de coleta e contagens de censura para tornar essas decisões auditáveis.

A validade externa é limitada a repositórios populares, públicos e que usam GitHub Actions. Os resultados não devem ser generalizados diretamente para equipes privadas, outros provedores de CI/CD ou sistemas com telemetria real de produção. Para a validade de conclusão, a multiplicidade dos 16 testes de RQ06 foi controlada por Holm, com tamanhos de efeito e intervalos de confiança por *bootstrap*, além dos valores de p. Ainda assim, os grupos C# e C++ têm só 15 repositórios, o que reduz o poder dos testes e alarga os intervalos; idade e número de contribuidores estão correlacionados, o que confunde seus efeitos; e o tipo do projeto não pôde ser testado, pois a amostra-ouro não se sobrepõe à amostra final.

6 Replicação

O repositório contém o código de coleta, geração do conjunto consolidado, notebooks de análise, tabelas intermediárias e figuras. A separação entre dados brutos, processados e resultados permite refazer cada etapa e inspecionar as decisões operacionais. Seeds, versões das dependências e comandos de execução devem permanecer documentados junto à entrega final.

7 Conclusão

A mineração de métricas DORA em repositórios open-source é viável, mas as medidas resultantes são sensíveis aos proxies disponíveis. Entre 21,7% e 37,7% dos repositórios mudaram de categoria sob definições plausíveis, e os dois CFRs apresentaram correlação praticamente nula. A taxa de retrabalho, com mediana de 14,3%, complementa o CFR ao contar a ação corretiva em vez da release presumivelmente falha. Velocidade e estabilidade se mostraram independentes pelo CFR de CI; a aparente relação entre elas pelo CFR de entrega vem, em grande parte, da própria definição. A idade do repositório é a característica mais associada à velocidade de entrega, e nenhuma característica testada se associa à estabilidade. Esses resultados recomendam explicitar as definições, publicar análises de sensibilidade e evitar interpretar categorias como atributos absolutos dos projetos.

Referências

[1] Forsgren, N., Humble, J. e Kim, G. (2018). *Accelerate: The Science of Lean Software and DevOps*. IT Revolution.

[2] DORA (2024). DORA's software delivery metrics: the four keys. `https://dora.dev/guides/dora-metrics-four-keys/`.

[3] Romano, J., Kromrey, J. D., Coraggio, J. e Skowronek, J. (2006). Appropriate statistics for ordinal level data: Should we really be using t-test and Cohen's d for evaluating group differences on the NSSE and other surveys? In *Annual Meeting of the Florida Association of Institutional Research*.

[4] Tomczak, M. e Tomczak, E. (2014). The need to report effect size estimates revisited. An overview of some recommended measures of effect size. *Trends in Sport Sciences*, 1(21):19–25.